CTO Final-Round Interview Practice

Software Engineer · Browser Automation · Bravebird.ai simulation

15 October 2026

- CTO Final-Round Interview Practice
  - How to practice
  - Round 1 — Ownership and project defense
  - Round 2 — Browser automation and reliability
  - Round 3 — Security and API/UI review
  - Round 4 — System design for the startup
  - Round 5 — Product judgment and behavioral questions
  - A compact scoring rubric

CTO Final-Round Interview Practice

Software Engineer — Browser Automation · Bravebird.ai interview simulation · 2026

Role-play premise: you are interviewing with the CTO of **Bravebird.ai, a browser-automation startup founded in 2026**. This is a challenging practice interview, not a record of questions from a real Bravebird.ai interview. Answer each question aloud before reading the evaluation notes. Use your own work history for behavioral answers; do not invent examples. Describe your ownership of this project and any AI assistance honestly.

How to practice

The interviewer should press for concrete evidence: which file, which test, what failed, what tradeoff, and what would change at production scale. Aim for a direct first answer, then explain one implementation detail and one limitation. For a system-design prompt, clarify users and workload before drawing components. For behavioral prompts, use a real situation, your own action, and a measurable result.

Round 1 — Ownership and project defense

1. Give me the two-minute version of this project. What problem does it solve, and what did you personally build?

Strong answer should cover: A JSON-configured Playwright runner; native GitHub password plus TOTP; positive session validation and reuse; form navigation and PDF capture without publishing; manifest and redacted failure evidence; production resolver plus offline mutation tests; local bounded queue/UI as extra credit. Distinguish the real GitHub acceptance run from synthetic fixture tests.

Follow-up: Which part would fail first if GitHub changed its release form tomorrow?

2. Walk me from `python -m automation run` to the PDF on disk. Name the files and the data passed between them.

Strong answer should cover: `__main__.py` loads `.env` and dispatches; `config.py` reads portal/input JSON and substitutes named tokens; `models.py` validates; `Runner` checks state/authentication, executes steps using `Resolver`, captures and validates the document, and writes the manifest. Mention where redaction enters.

Follow-up: Where would you put a new screenshot-capture action, and what would you test?

3. Why are portal instructions and business inputs separate JSON files? What belongs in each?

Strong answer should cover: Portal-specific origin, selectors, login, and action sequence belong in the adapter; target repository, tag, title, notes, and filename are business data. Separating them permits new input scenarios without changing code and permits selector fixes without editing business values. `load_config()` uses a narrow token grammar and never evaluates JSON as code.

Follow-up: What happens if an input contains `{{env.SOMETHING}}` or a required key is missing?

4. Explain exactly how the runner decides whether a saved session is still authenticated.

Strong answer should cover: Check the state envelope version/origin/schema, load it into a new context, open the configured private page in a separate tab, require multiple positive markers, verify account identity, optionally validate an authenticated endpoint/JSON. A 200 response alone is insufficient. On invalid state, discard the context and log in from a clean one.

Follow-up: Why do you use a separate tab? What if the account switches during the run?

5. Your evaluator corrupts the saved JSON. What happens? What if the JSON parses but the server revoked its cookie?

Strong answer should cover: Parse/schema errors mark it corrupt. Valid-looking but revoked state is marked stale after positive private-page checks fail. In both cases a fresh context clears stale cookies/local storage and performs full login. The manifest records the distinction. State is saved only after confirmed login.

6. How does the second-factor code arrive in this implementation? What happens when the code is rejected?

Strong answer should cover: TOTP is generated locally from a Base32 enrollment seed; optional IMAP polls a mailbox over verified TLS and filters sender, subject, date, and unique code. The runner retries only a clearly visible rejection, once, with a new code. Ambiguous submission/network results are not blindly replayed. Email device confirmation is not necessarily account-level 2FA.

Follow-up: What are the main risks of storing the TOTP seed in `.env`? What would you replace it with in production?

7. Why does a TOTP seed belong in configuration, but a six-digit code does not?

Strong answer should cover: A seed is the long-lived shared secret used by the authenticator and runner to derive a time-window code. A six-digit code expires quickly and is an output, not enrollment material. Store the seed in a managed secret store in production, scope access narrowly, rotate/re-enroll safely, and never log it.

8. How do you know the changed-input test changed the document, not just the output filename?

Strong answer should cover: The live acceptance manifest records distinct title/notes/tag/filename and the user reviewed the alternate PDF's visible content. Local fixture tests assert different rendered document content and hashes. Raw PDF hashes alone are not a sound equivalence test because Chromium can include changing metadata. A stronger automated live check would extract/normalize PDF text or compare a stable rendered-content digest.

Follow-up: Would you accept a test that asserts only that the input JSON changed? Why or why not?

9. What does the manifest let an operator diagnose? What deliberately does it leave out?

Strong answer should cover: Portal, redacted inputs, state/reuse/full login, pages, stages/timing/retries, locator outcomes, output bytes/hash, and failure reference. It avoids raw credentials, OTPs, raw console/page text, and raw exception messages. Evidence masks page text and attributes while retaining structure, so diagnostics are safer but less visually useful.

10. Tell me about a failure you encountered while building this and how you fixed it.

Use a real story: The GitHub TOTP flow and release-tag dialog changed across retries. The code had to recognize auto-submit after code entry and the current “Tag: Select tag” → “Create new tag” → `Tag *` sequence. Explain evidence used, selector changes, regression test, and live retest. Do not claim the early failing version worked.

Follow-up: What would you do differently to shorten the debugging loop?

Round 2 — Browser automation and reliability

11. Why resolve controls by accessible role and name first? Why not use CSS everywhere?

Strong answer should cover: Role/name and labels express user-facing meaning and tend to survive layout/class changes; CSS can be fast but brittle. The resolver then tries exact text/nearby field text, portal-specific structure, and CSS alternatives. Requiring two descriptions gives fallback options.

12. How does the resolver behave with nested iframes and an asynchronously rendered control?

Strong answer should cover: Repeatedly enumerate current frames until a bounded deadline; for each strategy, search every frame before moving to a weaker strategy. Record outcomes and tolerate detached-frame races. Return exactly one visible match; ambiguous matches fail rather than choosing `.first`.

Follow-up: What if the same accessible name occurs in the main page and an iframe?

13. Which failures retry? Why don't you retry every Playwright exception?

Strong answer should cover: Timeouts, known connection resets, 429/5xx, temporary locator unavailability, dependent-field readiness, and dropped IMAP connection can retry within a bound and backoff. Invalid configuration, ambiguity, rejected authentication, most 4xx, unsupported prompt, invalid PDF are final. Repeating a click/download/OTP could duplicate a side effect.

14. A click times out after the portal may have processed it. How do you avoid duplicate side effects?

Strong answer should cover: Treat outcome as ambiguous, do not blindly retry non-idempotent actions. Inspect page/manifest and use an idempotency key or state read if the portal offers one. Retries require explicit adapter opt-in and a recovery contract.

15. Your PDF capture succeeds but the process crashes before the manifest is written. How would you make this recoverable?

Strong answer should cover: Current manifest is written at completion and API job state is memory-only, so a crash can leave an orphaned artifact/job. Production changes: durable job records with state transitions, atomic artifact finalization, manifest checkpoints, job IDs in output paths, reconciliation of orphan files, and explicit at-least-once semantics.

16. Why block network requests and JavaScript in offline resolver mode?

Strong answer should cover: Make the test deterministic, prevent fixture pages from fetching data or mutating, and ensure no account/service is contacted. `srcdoc` makes frames reproducible. It validates locator robustness, not portal authentication or all browser behavior.

17. What exactly do the HTML mutations prove? What don't they prove?

Strong answer should cover: They prove the configured targets remain resolvable after class changes, sibling reorder, wrapper insertion, and iframe relocation on the saved fixture using the production resolver. They do not prove the live GitHub DOM will never change or every accessibility/selector case works.

18. How would you test that failure evidence contains no sensitive values?

Strong answer should cover: Seed synthetic credentials, OTPs, identity strings, and form values into fixture pages; intentionally break a step; inspect JSON, sanitized HTML, and screenshot source; assert secrets absent. Current screenshot comes from the inert sanitized copy. Add fuzz/property tests for attribute names and encoded secrets. Review data retention and access controls.

19. Why do PDF hashes differ for repeated runs with the same input?

Strong answer should cover: PDF metadata, timestamps, document IDs, or renderer variation can change bytes while visible content remains the same. Compare extracted text/layout or a canonicalized content representation. A hash is useful for integrity of one file, not semantic equality across independently generated PDFs.

20. What does `retry_safe` mean in this code? Is it enough for production?

Strong answer should cover: It permits retries only for adapter-declared safe actions; fill/select/wait are treated as idempotent. A boolean is a coarse model. Production should model idempotency/reconciliation per action and account for side effects that happen before timeout.

Round 3 — Security and API/UI review

21. What can an attacker do if they obtain `.state/github.json`?

Strong answer should cover: It contains browser cookies/storage that may grant account access until revoked/expired. Keep it out of Git, restrict permissions, encrypt the machine/disk, isolate accounts, rotate/revoke on exposure, avoid copying it into support bundles, and use managed encrypted secret/state storage in production.

22. The API is loopback-only and has a bearer token. Is it safe to expose on a LAN?

Strong answer: No. The implementation is a local developer interface. It has no TLS, multi-user authorization, durable audit, rate-limited identity model, or deployment hardening. Loopback binding and a token reduce local accidental access; they are not a production network boundary. A production service needs authenticated users, TLS, CSRF/CORS/host controls, secrets management, audit, isolation, and threat modeling.

23. What does the current job queue guarantee if the API process restarts?

Strong answer should cover: Nothing durable. Jobs and statuses live in memory, and one worker serializes them only while the process runs. A restart loses pending/in-memory status. Explain accepted-vs-completed states and design durable persistence before claiming exactly-once execution.

24. The API accepts JSON paths. How do you prevent path traversal?

Strong answer should cover: Reject absolute paths, `..`, backslashes, non-JSON files; resolve symlinks and require the result to remain under the project root. Limit request size and reject unknown body keys. Follow-up: review TOCTOU if an untrusted local user can replace files between validation and execution.

25. The UI uses `textContent` for API output and CSP. Why?

Strong answer should cover: Avoid interpreting job-controlled strings as HTML; restrict script/style/connect sources to same origin, no framing/base changes. Token is not put in localStorage. Still not a public multi-tenant UI; local users/processes and malicious browser extensions are in threat model.

26. Could a malicious website use a user's browser to submit a local job?

Strong answer should cover: Current API requires an unpredictable bearer header for POST/status and doesn't enable cross-origin CORS, which blocks straightforward browser reads. Production-grade localhost services should also validate Host/Origin, defend against DNS rebinding and CSRF, use short-lived tokens and bind only where intended. Do not represent current prototype as hardened for hostile local websites.

27. What is the privacy tradeoff in evidence sanitization?

Strong answer should cover: Masking all text and attributes protects unknown personal data but reduces ability to diagnose copy/layout issues. Improve with explicit user-consented capture scopes, synthetic repro fixtures, secure short retention, access controls, and separate metadata from content. Never default to storing raw authenticated pages.

Round 4 — System design for the startup

28. Design a service that runs browser jobs for 100 portals and 1,000 jobs per hour. Start with questions, not components.

Clarify: Is that average or burst? How many customer accounts? Which portals permit automation? Are jobs interactive, scheduled, or event-driven? What are latency/SLOs? Which documents are sensitive? How much concurrency can each portal tolerate? What is the retry and duplicate-work tolerance?

Strong design should include: API/authentication, durable queue, scheduler/worker pools, per-account session vault, per-portal adapter/version registry, browser isolation, rate limiting, cancellation, durable artifact storage, manifest/audit store, observability, and operator replay. Explain tenant isolation and backpressure.

29. How do you scale concurrency without corrupting sessions or overloading portals?

Strong answer should cover: Lock by account/origin, separate browser contexts and state per account, global and per-portal concurrency/rate limits, worker pools by capability, queue priorities, bounded retries, circuit breakers, and cancellation. Do not share one Playwright context across jobs or increase worker count blindly.

30. A portal changes its form silently. How do you detect and recover at fleet scale?

Strong answer should cover: Versioned adapters, contract tests on sanitized snapshots, locator miss/ambiguity metrics, stage failure rates, canary runs, alerting by adapter version, quarantine/circuit breaker, rollback, manual review, and evidence that does not leak user data.

31. Would you add proxy routing to this product?

Strong answer should cover: Only for a legitimate network requirement and with explicit customer authorization, documented egress, secret isolation, and compliance review. Do not use proxies to evade anti-bot controls or account restrictions. First solve portal permission, reliability, and account isolation. Measure whether proxy support is actually needed.

32. How do you reason about “exactly once” when a browser action can succeed before a worker crashes?

Strong answer should cover: Exactly-once is generally not achievable across an external portal without portal idempotency. Use at-least-once delivery plus idempotency keys where supported, state reconciliation, action ledgers, dedupe windows, and explicit job outcomes such as `unknown`/`needs_review` for ambiguous side effects.

33. What metrics would you put on a dashboard for this service?

Strong answer should cover: Queue depth/wait, run duration by stage and portal, login success/MFA failure, session reuse/staleness/corruption, locator fallback/ambiguity, retries, document validity/capture failures, worker/browser crashes, portal 429/5xx, cancellation, cost per successful job, and redaction-safe audit events. Avoid labels containing user identifiers or secrets.

Round 5 — Product judgment and behavioral questions

34. A customer asks you to automate a workflow that violates the portal's terms or bypasses a challenge. What do you do?

Strong answer should cover: Clarify authorization and terms, decline bypass/anti-bot evasion, explain safe alternatives such as an official API, approved integration, or manual step, and escalate legal/security uncertainty. Protect users and the company's access, not just the demo.

35. Tell me about a time you had to choose between shipping quickly and making a system safer.

Use your own example. State the risk, what you shipped or delayed, how you reduced risk, who you involved, and what happened. The interviewer is testing judgment and communication, not a memorized “always slow down” answer.

36. Describe a time a teammate disagreed with your design. How did you resolve it?

Use your own example. Show how you made assumptions explicit, sought evidence, changed your mind if warranted, and committed after a decision. Avoid blaming the teammate or claiming you always won.

37. You are on call and job failures spike after a portal deployment. What do you do in the first 15 minutes?

Strong answer should cover: Confirm blast radius and account impact, pause/quarantine failing adapter or retry loop, preserve sanitized evidence, inspect stage/locator/error metrics, notify stakeholders, avoid unsafe replays, roll back or switch to manual handling, then write a post-incident action plan.

38. What would you ship in your first 30 days at a small browser-automation startup?

Strong answer should cover: Learn customer workflow and permissions; map reliability/security risks; instrument baseline success by stage; fix the highest-impact failure mode; add adapter contracts and a safe rollout path. Avoid proposing a broad rewrite before understanding constraints.

39. What is one part of this project you would not defend as production-ready?

Strong answer: Name a real limitation. Examples: in-memory API queue/jobs; local bearer token and no production auth/TLS; no concurrent multi-account worker isolation; PDF equality is not semantic; live Gmail IMAP not validated; one GitHub adapter. Describe the smallest responsible improvement.

40. What question would you ask me as CTO?

Good questions include: What reliability target matters most to customers? Which workflows require official APIs versus browser automation? How do you decide when to stop retrying and escalate to a human? What customer data may workers inspect and retain? What does success for this role look like after 90 days? How does the team handle portal changes and security incidents?

A compact scoring rubric

Score each answer from 1 to 4:

- **1 — Vague:** gives slogans or guesses; cannot explain the implementation or risks.
- **2 — Partial:** understands the feature but misses failure modes or test evidence.
- **3 — Strong:** accurate, specific, explains a tradeoff, and cites a test or artifact.
- **4 — Senior-level:** does all of the above, anticipates follow-up risks, and proposes a staged improvement without overengineering.

For project questions, practice pointing to the relevant module: runner/session logic (`automation/runner.py`), providers (`automation/otp.py`), resolution (`automation/resolver.py`), validation (`automation/models.py` and `tests/`), and API/UI (`automation/api.py`, `automation/ui.*`). For every claim, distinguish live GitHub evidence, local fixture evidence, and planned production work.